A Nanomagnetic Logic Data Memory Design

Fernando Italo, Lorenzo dos Santos,
Luis Fernando Miki, Vinícius Reis Camillo

FEDERAL UNIVERSITY OF MINAS GERAIS (UFMG), BRAZIL

February 21, 2022

Abstract

Nanomagnetic logic (NML) is an efficient technology for transmitting binary information through magnetic fields of nano-sized magnets spatially disposed. In this work, designed a data memory using NML circuits and simulated it using the NMLSimV2 software. The memory cell contains a read/write command, a gate for input for the write command and the core logic circuitry. We performed simulations of read and write commands, and the simulation results showed that the memory cell behaved as expected.

I. Introduction

Computation has had a growing importance in our current society. The energy needed for all the devices involved is also becoming considerably large and, therefore, the development of new technologies to reduce the energy usage by computing processes could prove to be a great enhancement.

Nanomagnetic Logic circuits (NML) are one of the new technologies that serve such purpose. Since they work without the need for an electric current, the energy waste and heat generation in those systems are smaller. The fact that nanomaterials are used in these types of circuits also provides a second advantage – decreased device size.

Because of the physics involved in NML, common circuits and logic gates must be adapted to function properly. In this work, we aim to design a data memory cell and ensure it has the correct functionality, by simulating its behavior with the NMLSimV2 software.

II. The physics

A NML circuit is a 2D circuit that works differently for each direction. Each nanomagnet is a long and thin piece of magnetic material, with height of about 130 nm and width about 15 nm. This creates an anisotropy within the material, making a strong magnetic orientation only possible in the height direction. Therefore, despite the magnetic field having 3 components, only the one in the commonly assigned Y axis is used for the computation, and so each magnet can more easily be associated with a 1 or a 0 bit.

The spacial disposition of NML magnets is relevant when designing a circuit. The linkage between two nanomagnets can be either ferromagnetic or antiferromagnetic, and that depends solely on the direction of such linkage. Figure 1 presents the possibility for both connections and the respective induced magnetic orientation on the second magnet, provided the orientation on the first magnet. When the magnets are aligned in the X axis, the magnetic orientation is reversed. When the magnets are aligned in the Y axis, the magnetic orientation is preserved.

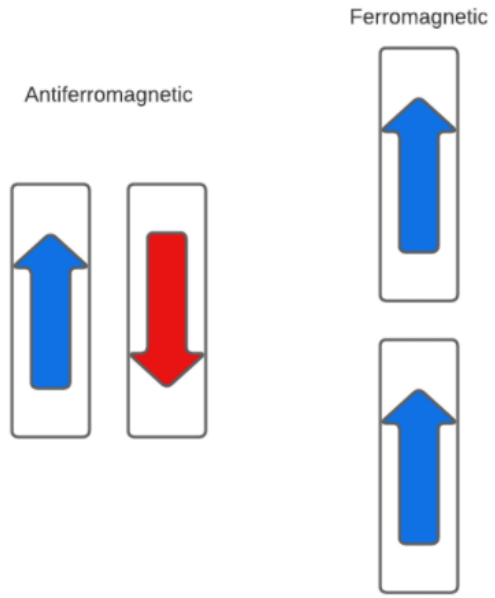

Figure 1: *Different possible arrangements of NML circuit components.*

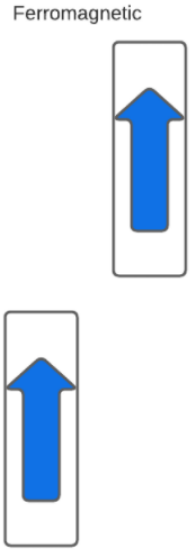

Figure 2: *Diagonal coupling of NML circuit components.*

If these were the only possible couplings, that would result in a major obstacle for many logic circuits, since the inversion signal would be directly related with the length of a part of a system in the X direction. Nevertheless, a diagonal coupling is also possible, as shown in Figure 2. This creates a ferromagnetic non-inverting propagation of the signal through the X axis, which enables a more design possibilities with this technology.

The shape of a magnet can also be used as a tool for designing a system. Using a bevel edge on the top half, it is possible to create an “AND” logic gate. That happens because a cut in the top part of the magnet makes it more difficult for the magnet to get the value “1”. Hence, when it is influenced by 2 magnets, having both magnets with opposite influences result in a negative magnetic field, which is comprehended by a 0. In an analogous way, having the bevel in the bottom part of the magnet allows the creation of an “OR” logic gate. Both configurations are shown in figure 3.

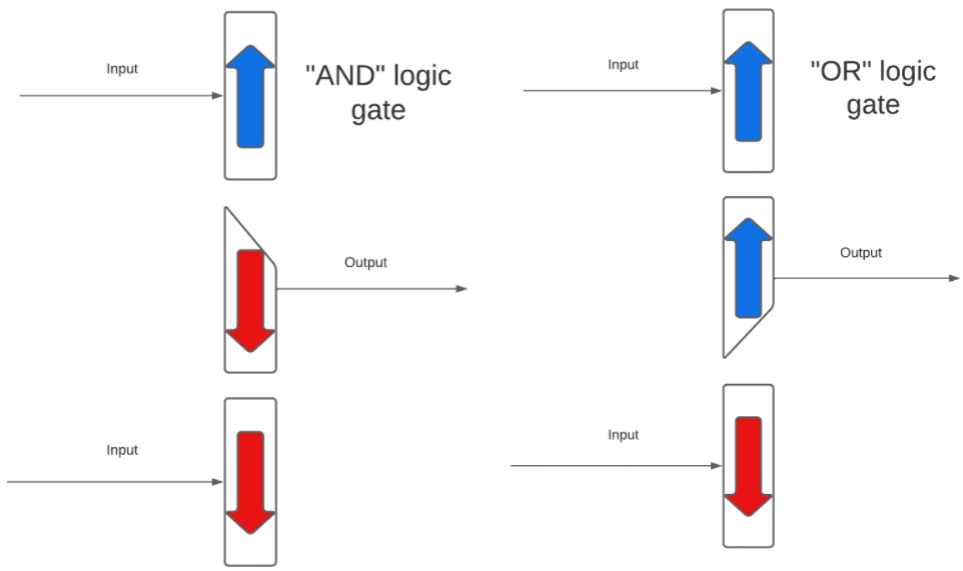

Figure 3: *"AND" and "OR" logic gates implemented in using NML circuit components.*

Gates and some elbow regions of a system can also provide for some undesired magnetic influence, after all, the system is projected for a magnet to only influence the magnet immediately after it. The bevels can also alleviate this undesired influence. Although, differently from its use in the logic gates, the bevel must be done in both the top part and bottom part of a magnet for this scenario. That is to prevent a prior influence. Cutting both the bottom and top part, the magnet stays neutral, and it stays just as likely to be magnetized upwards or downwards.

There are four clock phases in an NML system – reset, relax, switch, and hold. During the reset and switch phases, the magnet is in a transitory state, going from the relaxed state to the magnetic orientation determined by the signal in the hold phase. The hold phase is the one where it needs to keep the correct orientation to transmit the signal forward, for the next magnets, which are in the switch phase. This four phase system prevents the signal from traveling backwards, since it puts a much higher physical distance between a magnet in the hold phase and the previous switch magnet, diminishing the influence.

III. THE D/SR FLIP-FLOP IN NML

The first part for designing the memory was to design the D/SR Flip-flop which will be a component for the complete system. It can be divided into three parts: The read/write command, the input for the write command, and the core logic circuit. The complete system is shown in figure 4.

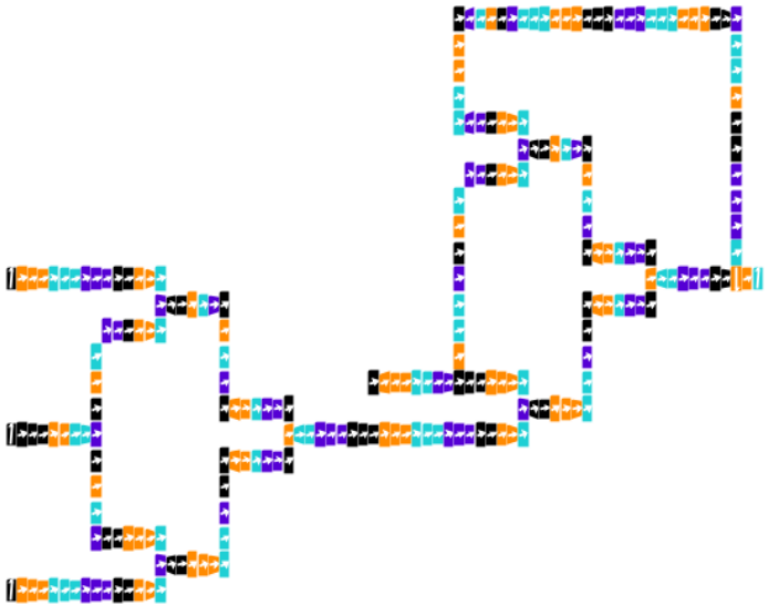

Figure 4: *D/SR flip-flop.*

The enable is the simplest part of the system. It's a signal input where 1 represents the "write" command and 0 represents the "read" command.

The input for the write command is a more complex system. It is composed of two "AND" gates, one "OR" gate and an inversor. There are also 3 bits of data that enter this system - the "input", the "set/reset", and the "select". "Select" is the command that decides which one of the entries should be used. Having the select as 1 transmits that 1 to the "AND" logic gate after the "input", allowing it to go forward. The inversor makes the select transmit a reversed signal for the "AND" gate after the "set/reset" entry. It is only when the select is 0 that the gate allows the "set/reset" entry to pass into the system. The signal for both gates reach a final "OR" logic gate, that allows any of the signals to pass to the core logic circuit. The system is represented in the figure 5.

Figure 6 shows the core logic circuit. It has the same components as the write command circuit but with a feedback loop, so that the written bit can be read and have the flip-flop work as a memory. The "input" entry is switched for the write command circuit, the "select" entry is switched for the read/write command, and the "set/reset" entry is switched for the feedback loop.

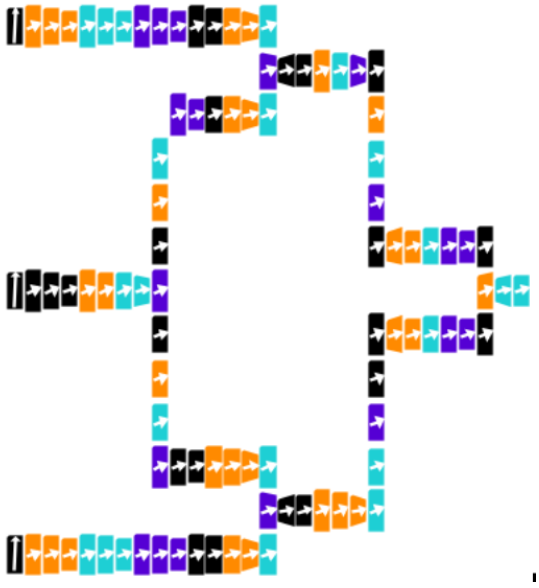

Figure 5: *Input for the write command.*

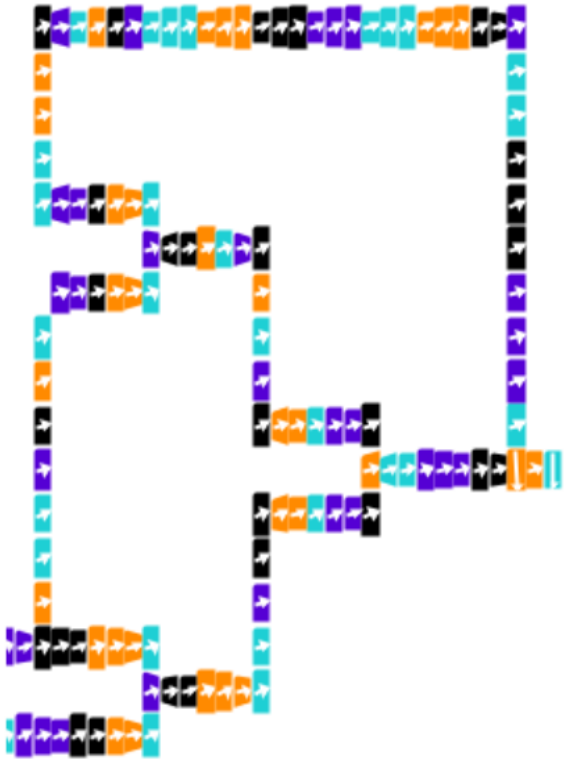

Figure 6: *Core logic circuit.*

IV. DATA MEMORY ORGANIZATION

In order to avoid crossing wires which multiplexing the outputs of each individual memory cell would cause for accessing the data memory like a structure assembled with 4 memory positions, 1-bit each. The presented memory was designed in a way that is actually a 4-bit register *M*.

The register *M* has four addresses used to modify each of its bits. The 2-bit *W_addr* signal selects the specific bit of *M* to be modified, in a writing operation case, for instance if *W_addr* has a value of '00', then indicating that Data_in needs to deliver its value to *M[0]* memory cell (the leftmost flip-flop in figure 7). In this way, the data is read in four parallel wires.

In the reading operation, we simulate the *W_addr* signal setted to be '00'. You can see that it will enable the write on the first flip-flop. To bypass this, the *Sel* signal is set to 0 and the *S/R* gets the stored value, avoiding any changes on the memory value. This explains the dashed wire above the leftmost flip-flop in figure 7. Of course, a write/read signal that goes to all cells would be more appropriate, but like mentioned before, due to limited resources and specific simulation purposes, we simplified this memory structure to avoid the wire crossings.

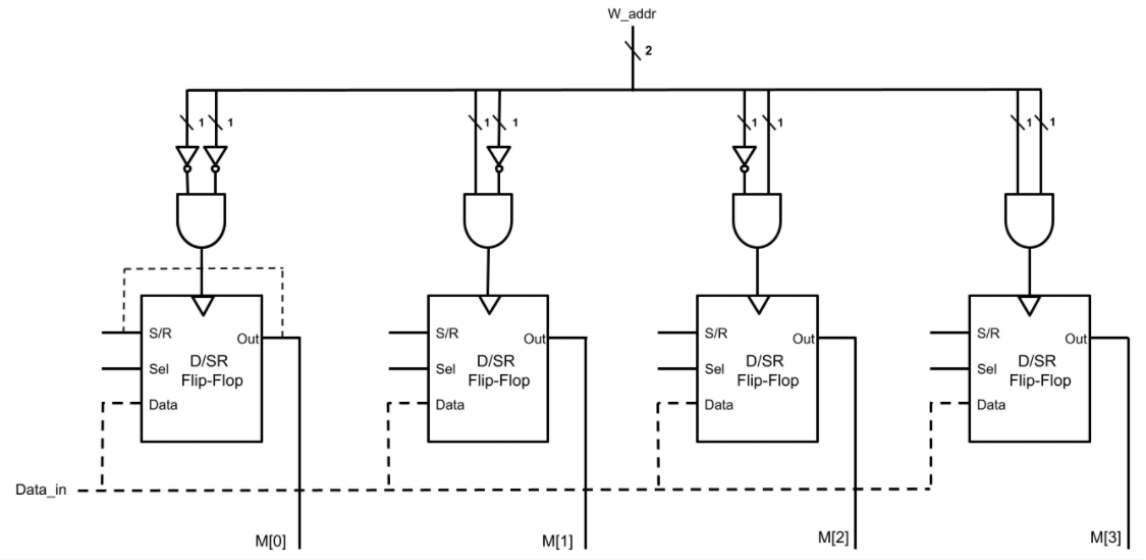

Figure 7: *Memory structure design.*

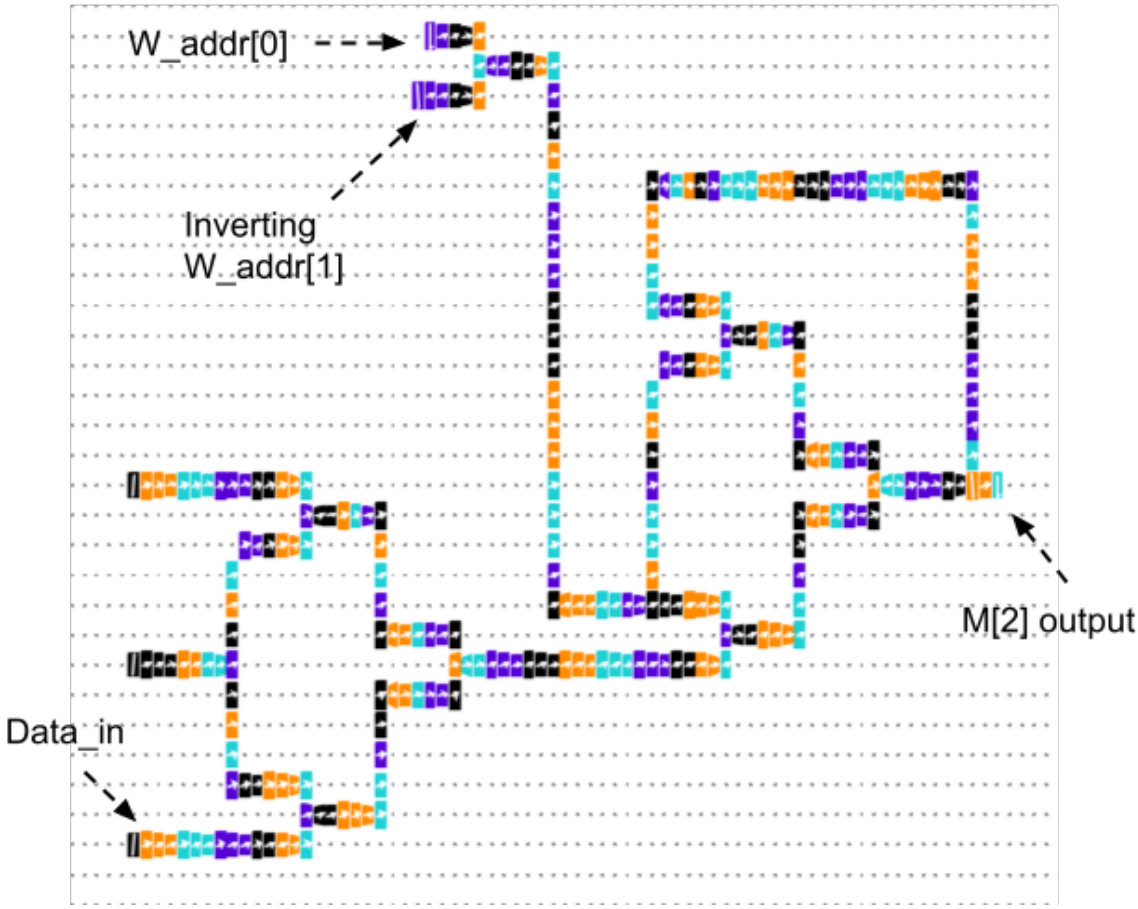

Figure 8: *M[2] memory cell.*

Figure 8 illustrates the design of the *M[2]* memory cell in figure 7. It is possible to see that the enable signal is translated as AND of the *W_addr* signal, but inverting its second bit. The same logic of inverting or not the *W_addr* signals then ANDing them, is added to the parallel memory cells to create an addressable design by concatenating the four flip-flops with the *W_addr* and *Data_in* wires.

V. SIMULATION RESULTS

After asserting the correct functioning of the D/SR FLIP-FLOP, the next step was to simulate the concatenated data memory design shown in figure 7. We present in this section a write operation and a reading operation simulation result.

Table 1 shows the parameters used to simulate the reading operation and the result M[X]_out.

N	M[N]_Stored	S/R	Sel	Data_in	M[N]_Out
0	0	M[0]_stored	0	X	0
1	0	X	X	X	0
2	0	X	X	X	0
3	1	X	X	X	1

Table 1: *Reading simulation input parameters.*

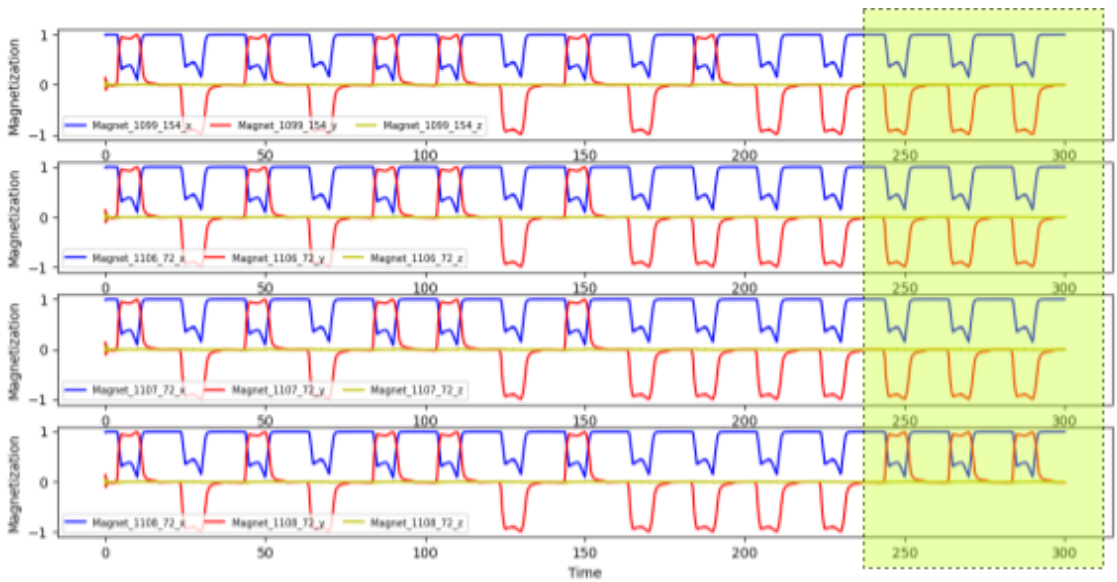

Figure 9: *Reading simulation waveform output.*

Also, the W_addr signal input is set '00', indicating a reading operation. Analyzing the waveform shown in figure 9, the green highlighted rectangle shows that a safe result free from oscillations is obtained after around 250 nanoseconds from the start of the simulation. This is because of the 12 clocking cycle's from the furthest input to the output.

From up to bottom, each waveform represents M[0], M[1], M[2], M[3], respectively. We can see that only the M[3] y component (red) is delivering the value '1'. This results agrees with the previous stored value 0001 for the M register. For this simulation, the necessary computation time was 4069857 milliseconds and a 18556 KB of memory were used on a Intel Core i5-7200U processor.

In table 2, there are the parameters used to simulate the writing operation and the result M[X]_out. The W_addr signal input is set '01', indicating a write on M[1].

N	M[N]_Stored	S/R	Sel	Data_in	M[N]_Out
0	0	X	X	1	0
1	0	X	1	1	1
2	0	X	X	1	0
3	1	X	X	1	1

Table 2: *Writing simulation input parameters.*

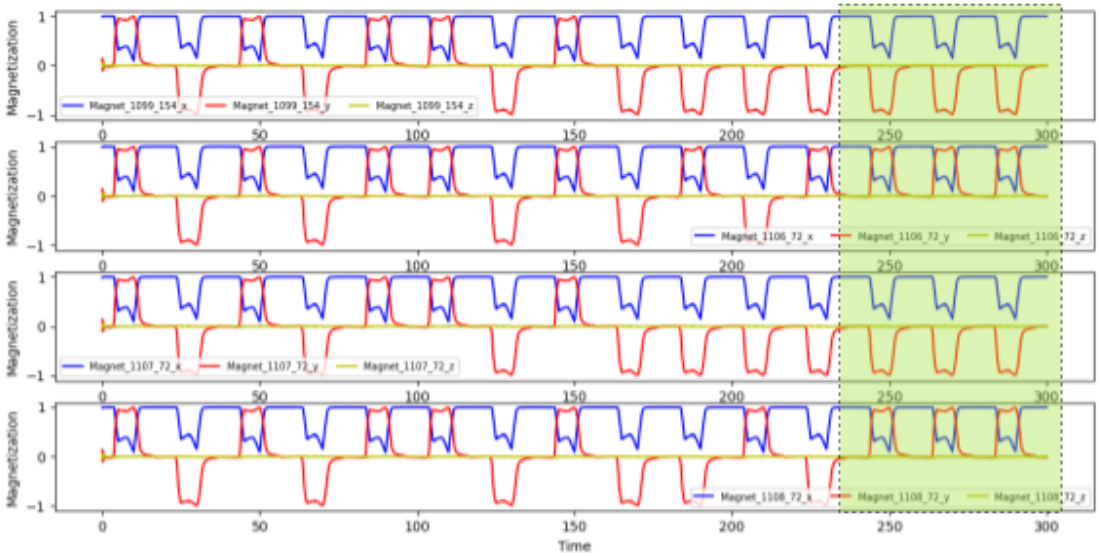

Figure 10: *Writing simulation waveform output.*

From the results from figure 10, the stored value in the M register was the output of the previous reading operation '0001'. Next we can see that the M register is now delivering a '0101' value due to write operation on M[1]. For this simulation, the necessary computation time was 4127354 milliseconds and 19260 KB of memory were used.

VI. Conclusion

It was possible to develop a data memory using a NML circuit. The simulation showed that the system would work correctly, and that the signal would need around 250 nanoseconds to travel the circuit. The crossing wires has shown itself to be a challenge for a 2-dimensional system, but that can be avoided in a physical model by using more layers in the third dimension. In that case, it's worth the attention to allow enough distance to prevent magnetic field interference between two layers.

References

1. E. Fazzion, O. L. H. M. Fonseca, J. A. M. Nacif, O. P. Vilela Neto, A. O. Fernandes and D. S. Silva, "A quantum-dot cellular automata processor design," 2014 27th Symposium on Integrated Circuits and Systems Design (SBCCI), 2014, pp. 1-7, doi: 10.1145/2660540.2660997.

2. L. A. L. Freitas, J. G. N. Rahmeier and O. P. V. Neto, "Shape Engineering for Custom Nanomagnetic Logic Circuits in NMLSim 2.0," in IEEE Design Test, vol. 38, no. 4, pp. 85-93, Aug. 2021, doi: 10.1109/MDAT.2020.3043381.